Document Scanner

Feature Overview

Field	Value
Feature name	Document Scanner
Status	Phase 1 complete; Phase 2 imaging complete; on-device edge detection and perspective correction added (Build 32), detection accuracy and auto-capture timing tightened (Build 33), outline steadied on Android and a capture confirmation added (Build 34), review-screen toggle between the auto-cropped scan and the original photo added (Build 35), searching viewfinder and a more forgiving hold added (Build 36), hold loosened further for iOS (Build 37), readable full-resolution scans, a quicker grab and a pinned save bar with exit confirmation (Build 38), auto-detect restored with a Build 37-style camera open and a paged review card (Build 39), fewer false grabs and better detection on busy backgrounds with a paper-edge check (Build 40)
Phase	Phase 1 (core) + Phase 2 (enhanced imaging) + Builds 32 to 40 (document intelligence)
Priority	High

Description

Browser-based document capture with three ways to add content: (1) scan with the device camera, (2) upload an existing file (PDF or image), or (3) choose an existing photo from the device gallery. Camera and gallery images can be enhanced (rotate, crop, brightness, contrast), combined across multiple pages, and saved as an image or a combined PDF. An uploaded PDF is saved as-is. Designed to work inside an iframe within the GoodBarber mobile app shell.

Requirements

Core (Phase 1)

- [x] Access device camera via browser APIs (getUserMedia)
- [x] Capture document image from camera feed
- [x] Basic image enhancement (contrast, brightness adjustment)

- [x] Edge detection (implemented in Build 32: on-device live edge detection with an auto-capture)
- [x] Generate PDF or image output from the captured scan
- [x] Save scanned document to cloud storage (S3) with metadata stored in the database
- [ ] Offline capture and sync (deferred - see Offline Support below)

Enhanced (Phase 2)

- [x] Multi-page document scanning (combine into a single PDF)
- [x] Interactive crop (draggable rectangle with corner handles)
- [x] Rotate in 90-degree steps
- [x] Brightness and contrast sliders (live preview)
- [x] Automatic perspective correction (implemented in Build 32: on-device deskew to a flat rectangle, with a one-tap “Use original” and the manual crop retained as a fallback)
- [ ] Document type detection (not implemented)
- [ ] OCR text extraction (stretch goal - not implemented)

Implemented Behavior (Phase 2)

- There are three ways in. The center scan button routes to /scan?pick=camera and opens straight into the camera with no intermediate screen. The Upload item in the bottom bar does not open the scanner at all: it owns a hidden image-only file input, so tapping it opens the phone’s native picker on top of the current screen. “Scan into this vault” from inside a vault opens /scan with no pick param and still shows the two-option chooser (“Scan with camera” and “Upload a file or photo”), which is the only path that accepts a PDF.
- Upload handoff: a File cannot travel in a URL, so a photo picked from the bottom-bar Upload input is parked in a small in-memory module (lib/pending-upload.ts, single slot, cleared on read) and the app routes to /scan?pick=pending, which takes the file and goes straight to the review and edit screen. Canceling the picker does nothing and leaves the user where they were.
- Accept lists matter on Android. Chrome and Samsung Internet map the accept list onto an Android intent, and the photo gallery picker is only used when the list is image types only. A non-image type such as application/pdf, or a bare extension the system cannot resolve to a MIME type (.heic and .heif are the usual offenders), makes the browser fall back to the generic Camera / Camcorder / Files chooser with no gallery. The Upload path and the in-camera gallery shortcut therefore use a plain accept=”image/*” with no extensions. Only the in-vault upload input carries application/pdf, because that path is meant to reach the file browser.
- Upload a file (in-vault path) accepts a PDF or an image. An image goes into the normal edit flow. A PDF skips the imaging tools and is shown as a “PDF ready to save” review, then uploaded as-is (no re-encoding).
- Cross-platform device handling (iOS and Android): iPhone HEIC/HEIF photos are decoded to JPEG in the browser (heic2any) before editing, so they display and save correctly on every platform. This still works with the image-only accept list, because the operating system reports HEIC as an image type and the conversion runs after selection. File type is detected from both the reported MIME type and the filename extension, because Android file managers and content-URI pickers often report an empty or generic type. A “Preparing your photo” overlay is shown while a large photo is decoded.
- The camera view has exactly three controls: Close scanner (top left), Choose from gallery (bottom left) and Capture (center). Close is a single-step exit: it stops the camera stream and replaces the route with /home, so one tap returns to the vaults. There is deliberately no second close control. The no-camera fallback offers upload, a camera retry and a way back to the chooser.

- The scanner has staged flow: chooser, loading, camera, no-camera fallback, edit, saving.
- Edit stage supports rotate left/right, interactive crop with corner handles, and brightness/contrast sliders. Applying a crop flattens the current rotation, filters and crop into a fresh working image.
- Multi-page: each captured page is baked into a page list shown as a thumbnail strip. Pages can be removed, and “Add page” returns to the camera.
- Save output: a single page saves as a JPEG (mime image/jpeg). Multiple pages, or the “Save as PDF” toggle, save as one PDF (mime application/pdf) built with jsPDF (A4, one image per page, fit with margin).
- Upload path is unchanged in shape: request a presigned URL (now also sending fileSize for the quota check), PUT the blob with a matching Content-Type, then POST metadata to /api/documents.
- The file-upload fallback (for devices without a usable camera) is preserved.

Technical Notes

Camera Access

- Uses `navigator.mediaDevices.getUserMedia()` for camera access.
- Requests the rear-facing camera by default on mobile: `{ video: { facingMode: "environment" } }`.
- Provides a file-upload fallback for devices without a usable camera.

Image Processing

- Canvas API for image manipulation. Rotation is applied to a canvas, then the crop sub-rectangle is taken, then brightness/contrast are applied via the canvas filter.
- Helpers live in `lib/imaging.ts`: `loadImage`, `renderProcessedPage`, `buildPdfFromImages`, plus the `CropRect`, `PageEdit` and `ProcessedPage` types.

HEIC/HEIF Handling (iOS)

- iPhones capture photos as HEIC by default. Browsers cannot decode HEIC on a canvas (and Android/desktop cannot display it), so a selected HEIC/HEIF file is converted to JPEG in the browser with `heic2any` (dynamically imported, so the decoder only loads when it is actually needed) before it enters the edit flow.
- After conversion the rest of the pipeline is unchanged: the JPEG is edited and saved exactly like any other image.

PDF Generation

- Uses jsPDF (client-side) to build a single PDF from the captured page images.
- One image per A4 page, scaled to fit with a margin.

PDF Handling in Viewers

- Because documents can now be application/pdf, the document viewer and the folder grid show a PDF placeholder icon plus an open/download action instead of a broken image. Downloads use the correct file extension (.pdf or .jpg).

Iframe Considerations

- Must work inside an iframe (GoodBarber context).
- Camera permissions must propagate through the iframe: requires the `allow="camera"` attribute on the iframe element.

- Note: the live app opens the vault first-party (breaking out of the embed) so that camera, cookies and login work reliably in the in-app browser. See build_state.md Builds 9 and 10.

Offline Support (deferred)

- Offline capture, local queueing and background sync are not implemented in Phase 1 or 2. This remains a design goal for a later phase.

Browser Compatibility

- iOS Safari: highest-risk area for camera in an iframe. The first-party break-out approach mitigates this.
- Android Chrome: primary target.
- Desktop browsers: secondary, for testing and demo.

Open Questions and Risks

Risk	Severity	Notes
iOS iframe camera permissions	Critical	Mitigated by opening the vault first-party from the GoodBarber home. Re-test after each redeploy.
Image quality on lower-end devices	Medium	Camera resolution and processing power vary.
PDF file size	Low	Large images produce large PDFs. Export uses a “fast” compression path.

Dependencies

- Cloud storage setup (S3 bucket, upload API)
- Database schema (documents table with metadata)
- Authentication (user ID for document ownership)

Capturing vault fields at save time (Build 25, Phase A)

The save step now renders the selected vault’s fields as inputs (text, number, date picker, or choice dropdown) below the vault chooser, so document details are captured at the moment of saving. Values are sent with the document create request, sanitized against the vault’s field definitions, and stored as the document’s metadata plus a derived search string. Fields shown are driven by the chosen vault; switching the vault switches the fields. See files.md for the full field model and the custom field builder.

On-device edge detection and perspective correction (Build 32)

The web scanner gained document intelligence that runs entirely on the device, with no third-party scanning service and no per-scan fees.

- Live edge detection: while the camera is open, the app watches the feed and draws an outline around the document it finds. The outline turns to the accent color once the whole document is well framed, and a hint pill guides the user. See Build 33 below for the current wording and the hold indicator.
- Auto-capture: once the framing is held steady and good, the app captures on its own. A manual shutter is always available too. The timing rules were rewritten in Build 33.
- Auto-detect toggle: a control on the camera bar switches the live detection and auto-capture on or off. It is on by default; turning it off gives a plain manual shutter with the static framing guide.
- Perspective correction (deskew): the captured document is warped to a flat, straight-on rectangle, so a photo taken at an angle comes out square. The review screen shows an “Edges detected and straightened” badge with a one-tap “Use original” to revert to the untouched photo.
- Uploaded and gallery photos are straightened too: a snapshot of a document taken outside the app is run through the same correction on import, with the same “Use original” undo.
- Graceful fallback: the computer-vision runtime (OpenCV.js plus jscanify) is loaded lazily from a CDN. If it is blocked or slow inside the in-app browser, the loader times out quietly, the overlay simply does not appear, and the plain manual capture, crop, rotate and brightness/contrast tools all still work. Nothing in the save, multi-page or PDF flow changed.

Technical notes (Build 32)

- New helper `lib/doc-scan.ts`: lazily injects OpenCV.js (WASM) and jscanify from pinned CDN URLs, waits for the runtime to be ready, and exposes `detectCorners` (find the document’s four corners), `cornersAreGood` (is the framing large and rectangular enough to auto-capture), and `correctCanvasToDataUrl` (perspective-warp to a flat rectangle). Everything is best-effort and never throws into the UI.
- No new package dependency and no build weight added to the core bundle: the CV runtime is fetched at runtime only when the scanner is used, and only if reachable.
- `components/scanner/scanner-screen.tsx`: a throttled detection loop (using a small downscaled canvas for speed) drives the live overlay and the auto-capture, and the capture and photo-import paths apply the perspective correction. The static framing guide is kept for when detection is off or has no result yet.

Detection accuracy and auto-capture timing (Build 33)

The first release of the live detection fired far too eagerly: opening the scanner produced an outline that jumped between arbitrary shapes and an automatic capture of whatever happened to be in view within about a second. Build 33 rewrites what counts as a document and when the app is allowed to capture.

What was wrong

The underlying library returns the largest edge blob in the frame and derives four extreme points from it, with no check that the shape is actually a rectangle. Anything with an outline (a desk, a hand, a shadow, the frame border itself) was therefore handed back as a “document”. The acceptance test

was also loose enough to allow a shape covering almost the entire frame, and the dwell before capturing was only about a second from the moment the camera opened.

What changed

- A candidate is now only accepted if the traced outline genuinely simplifies to a convex four-sided shape at a tight tolerance. Paper has straight edges, so its outline folds into four corners almost immediately; rounded and ragged shapes only do so once the tolerance is loosened, and those looser passes were removed.
- The shape then has to behave like a sheet of paper: it must cover between 22 and 90 percent of the frame, keep every corner clear of the frame edge, have opposite sides and diagonals of similar length, have all four corner angles between 62 and 118 degrees, sit within a sensible width-to-height range, and fill its own four-cornered outline.
- Capture timing is handled by a dedicated gate. There is a 1.2 second warm-up after the camera opens during which nothing can be captured (the outline still draws, so the user sees it working), then the document must be held steady for a further 1.4 seconds across at least 6 detections. The earliest possible automatic capture is therefore about 2.6 seconds, and only for something that has stayed put.
- Steadiness is measured two ways: movement between consecutive frames and total drift from where the document was first seen. A slow pan accumulates drift and resets the countdown instead of sliding into a capture, and a detection that flickers in and out resets as well.
- The hint pill now reads “Point at a document to detect its edges”, then “Line up the whole document”, then “Document found, hold steady…” with a progress bar that fills over the hold. The capture is visible in advance rather than a surprise.
- The drawn outline is eased between frames so it settles onto the document rather than flicking about, and snaps straight over if the document genuinely jumps.

Technical notes (Build 33)

- `lib/doc-scan.ts` : `detectCorners` no longer uses the library’s corner-point helper. It validates the contour itself with an approximation sweep limited to tight tolerances, a convexity test and a solidity band, and orders the corners consistently. `cornersAreGood` was rewritten with the geometry tests above. New helpers `cornersDrift` and `blendCorners` support the steadiness check and the overlay smoothing. All tuning values sit in one labeled block at the top of the file.
- New `lib/doc-scan-gate.ts` : a small, pure state machine holding the warm-up, hold and drift rules, separated from the React code so the timing can be tested directly.
- `components/scanner/scanner-screen.tsx` : the detection loop now feeds the gate and captures only when the gate says so, and reports hold progress to the hint pill.
- Verified before release: the gate logic passes 12 of 12 simulated scenarios, including that a full-frame shape never captures, that 30 sessions of random shapes produce no capture at all, that a genuine steady document captures at 2.6 seconds, that a moving or flickering document never captures, and that slivers, rhombuses and edge-touching shapes are rejected. The detection itself was run against 8 camera-style frames through the real computer-vision runtime, passing all 8: two real documents are still detected, while a cluttered desk, a plain wall, a large irregular pale blob, an evenly lit bright frame, a keyboard and a document running off the frame edge are all correctly rejected (the blob and the cut-off document both used to pass).
- Not yet checked on hardware: the live camera behavior still needs a spot-check on a real device in the in-app browser after deploy, since webview performance is device-dependent.
- No schema, data or package dependency change.

Steady outline on Android and capture confirmation (Build 34)

Testing on a Samsung phone (Samsung Browser, the default engine behind the GoodBarber in-app browser on that device) showed the live outline flicking between arbitrary shapes while the camera hunted for a document. The same build ran cleanly on iOS Safari. Separately, on iOS the automatic capture was so quick and silent that the user was moved to the naming fields with no sense that a scan had just been taken.

What was wrong

Build 33 made the capture rules strict, and those rules held: the wild outlines on Android never produced a false capture, and the on-screen hint correctly read “Line up the whole document”. The problem was that every candidate shape the detector returned was still drawn, whether or not it passed the quality tests. Android camera feeds are noisier and their exposure hunts more, so far more junk candidates appear per second than on iOS, and the overlay looked chaotic even though the logic underneath was behaving.

What changed

- The outline is now only drawn once the shape is worth showing: either it passes the full quality test, or the same shape has been seen in roughly the same place for three consecutive detections. One-off junk candidates are simply never painted.
- When a document briefly drops out of detection, the last good outline is held for half a second instead of vanishing, so the overlay no longer blinks on a momentary miss.
- A capture confirmation was added. On capture, automatic or manual, the frame freezes, a white shutter flash fades out, and a tick with “Scan captured” and “Preparing your document…” holds for about nine tenths of a second before the details step opens. The straightening still runs behind that confirmation, so it costs no extra time in practice.
- The detection and capture rules themselves are untouched from Build 33. Only what gets drawn, and what the user sees at the moment of capture, changed.

Technical notes (Build 34)

- `components/scanner/scanner-screen.tsx` only. `lib/doc-scan.ts` and `lib/doc-scan-gate.ts` are unchanged, so the Build 33 detection and gate test results still stand.
- New draw gate in the detection loop: `cornersDrift` compares the current candidate with the previous raw candidate, and a counter requires three agreeing frames before a shape that has not passed the quality test is drawn. A grace period of 500 ms holds the last outline when detection misses.
- Capture confirmation state (`captureFlash` , `flashOn`) drives a frozen-frame overlay above the camera view; the shutter flash is a white layer fading over 500 ms and the confirmation holds for 900 ms before the review step loads.
- Verified in the browser against a synthetic camera feed: with auto-detect on, a deliberately busy, constantly changing scene produced no outline at all across 70 samples over 14 seconds and no false capture, while a steady document was outlined, auto-captured after 1.7 seconds and straightened. The confirmation appeared 111 ms after the shutter and held 854 ms before the details step opened.
- Not verified on hardware: the fix targets Samsung Browser on Android, which cannot be run from the build environment, so the outline behavior still needs a spot-check on the reporting device after deploy.

Review-screen toggle between auto-cropped and original (Build 35)

After a scan is captured, the details step shows the straightened (perspective-corrected) image with a “Use original” button, so a user can fall back to the untouched photo if the automatic crop misjudged the edges. Two problems were reported. First, once “Use original” had been used there was no way back to the auto-cropped version short of rescanning. Second, if a user chose “Use original”, went back, and then scanned a new document, the review screen showed the previous original rather than the newest auto-cropped scan.

What was wrong

The original and the corrected images were not both retained. Switching to the original replaced the working image and left no reference to the corrected version, so there was nothing to switch back to. Separately, the stored original was not always cleared when leaving the review screen, so a fresh scan could still find and display the earlier original.

What changed

- Both versions are now kept side by side after a capture: the auto-cropped (straightened) image and the untouched original are held in memory together.
- The review screen toggles both ways. When the straightened image is shown, the badge reads “Edges detected and straightened” with a “Use original” button. After tapping it, the badge reads “Showing the original photo” with a “Use auto-cropped” button that switches straight back. Nothing is lost either way, and the toggle only appears when a straightened version actually exists.
- Every exit from the review screen now clears both stored versions: retake, going back to the chooser, applying a manual crop, and starting a PDF review all reset them. A new scan therefore always starts on its own newest auto-cropped image, never a leftover original from a previous scan.

Technical notes (Build 35)

- `components/scanner/scanner-screen.tsx` only. `lib/doc-scan.ts`, `lib/doc-scan-gate.ts`, `lib/auto-lock.ts` and `components/security/lock-provider.tsx` are unchanged, so the Build 33 detection and gate test results and the Build 34 auto-lock behavior still stand. No schema, data or package dependency change.
- The corrected and original images are held in two refs (`correctedDataUrlRef`, `originalDataUrlRef`). A `setVersions` helper stores both and marks whether a straightened version exists; `showingOriginal` tracks which one the review screen is displaying. `loadSourceFromDataUrl` is now purely a display call with no hidden side effects, so switching views never mutates the stored versions.
- `capture` and the upload/gallery ingest path store both versions and display the corrected one; `useOriginalImage` and `useCorrectedImage` only change which stored version is shown. `resetWorking` (used by retake and back), `applyCrop` and the PDF review path clear both refs and reset `showingOriginal`.
- Verified in the browser at phone width (390 px) against a synthetic camera feed: after an auto-capture the badge read “Edges detected and straightened” with “Use original”; tapping it showed “Showing the original photo” with “Use auto-cropped”; tapping that returned to the straightened scan; and after switching to the original then using Retake and rescanning, the new scan showed the auto-cropped version, not the previous original. Type check clean, production build succeeds, auth and session smoke tests pass.

- Not verified on hardware: behavior should still be spot-checked on the reporting device after deploy.

Searching viewfinder and a more forgiving hold (Build 36)

Two things were reported after Build 35 went live. The viewfinder opened with a square white box that did not feel document shaped, and in automatic mode a fixed box is misleading anyway because the app is searching for the document rather than asking the user to line it up inside a frame. Separately, iOS found a document, said “hold steady”, then kept dropping out and starting again, while Android held fine.

The viewfinder

- With auto-detect on there is no box. Four gold corner brackets set at document proportions breathe gently and a soft line sweeps down the frame, which reads as “looking for your document” rather than “put it here”. The moment real edges are found, the brackets and the sweep disappear and the gold outline takes over. Both animations are disabled when the device is set to reduce motion.
- With auto-detect off the guide is a document-shaped frame (A4 proportions, roughly 1:1.414) instead of the old square.
- Neither guide is drawn while the camera is still opening, so nothing flashes on screen before the picture appears.

The hold on iOS

The hold used to be all-or-nothing: any frame in which the detector failed to return a usable shape destroyed the hold and started it from zero. On a device where the detector misses a frame now and then, which is what iOS was doing, the hold could never complete even with a perfectly still document. The hold is now forgiving:

- A missed frame costs one earned frame instead of resetting to zero, and up to three consecutive misses are tolerated before the hold is genuinely abandoned. A frame is still only earned by a real detection, so a detector that alternates hit and miss nets nothing and never completes a hold.
- The stale window, which is how large a gap is allowed before the hold is dropped outright, went from 0.7 to 0.9 seconds to suit a slower detection cadence.
- Per-frame steadiness went from 3 to 4.5 per cent of the frame diagonal, and drift is now measured against a smoothed version of the previous corners rather than the raw last frame, so detector jitter is no longer read as hand movement.
- Cumulative movement across a hold was loosened only from 5 to 6 per cent. A larger figure was considered and rejected: at 9 per cent a slowly panning document would still reach six frames and capture, which would weaken genuine detection.

The rule that a document must genuinely be detected before an automatic capture is unchanged. A capture still needs the full warm-up, the full 1.4 second hold and six earned detections.

Feedback while holding

- The progress bar gives ground on a missed frame instead of freezing, so a hold that is struggling looks like it is struggling. The outline and the wording stay steady through a dropped frame, only the bar moves.
- After three broken holds within eight seconds the hint changes to “Hold still, or tap the shutter to capture”, which offers the manual route rather than leaving the user waiting.

Diagnostics overlay

Adding `?debug=1` to the scan URL shows a small panel in the corner of the viewfinder while the camera is open: video and detection canvas sizes, detections per second, average detection time, the percentage of frames in which a document was found, earned frames out of six, per-frame and total drift, the gate's current reason, and the number of broken holds. It exists so the iOS behavior can be measured on a real iPhone rather than inferred, since real handsets cannot be run from the build environment. It is off unless the flag is present.

Technical notes (Build 36)

- `lib/doc-scan-gate.ts`: the miss path increments a miss counter and decays earned frames by one instead of resetting; a full reset happens only with no anchor, more than three consecutive misses, or a gap beyond the stale window. Tolerances are 4.5 per cent per frame and 6 per cent cumulative, the stale window is 900 ms, and the drift reference is an exponentially blended version of the last accepted corners. The feedback object now also carries the gate's reason, whether a hold has just broken, earned frames, held time and both drift figures, which is what the diagnostics panel reads.
- `components/scanner/scanner-screen.tsx`: the old static square guide is replaced by a document-shaped manual frame (auto off) and an animated searching indicator (auto on, no outline yet), both gated on the camera stage. The hint pill gained the struggle message and shows the bar whenever there is progress rather than only on a good outline. Inside the grace window a missed frame keeps the outline and wording but takes the decayed progress value, which is what makes the bar drain. The debug flag is read once from the URL and a 500 ms interval publishes the figures.
- `app/globals.css`: the breathing and sweep animations, both disabled under reduced-motion preferences.
- `lib/doc-scan.ts` (the detector itself) is unchanged, so the Build 33 detection results still stand and Android detection behavior should be unaffected. No schema, data or package dependency change.
- Verified: gate unit test 15 of 15, including "occasional blip still captures" (capture at 2.8 s) and "a real loss restarts the hold", with the flickering, moving, full-frame and random-shape guards all still refusing to capture. In a real browser at 390 px against a synthetic camera feed: brackets and sweep with nothing in view and no white frame, a document-shaped frame with auto off, first outline at 200 ms with the searching indicator gone in the same frame and capture at 2.1 s, a feed blanking detection every 700 ms still capturing at 1.7 s, the struggle hint at 5.7 s with no false capture, and the bar observed draining (15, 30, 33, 50, 67, 50, 67, 83, 67, 83 per cent) before capture. Type check clean, production build succeeds, auth and session smoke tests pass.
- Not verified on hardware: real iPhone and Samsung Browser devices are unavailable in the build environment, so the iOS fix needs a spot-check on the reporting device after deploy.

A more forgiving hold for iOS (Build 37)

Build 36 made iOS much better, but the "hold steady" stage still restarted more often than it should on the iPhone. The cause was not the tolerance figures alone: a single frame judged unsteady threw the whole hold away and started counting from zero. On a handset where the detector places the corners slightly differently from frame to frame, that hold could never finish.

A frame that lands wide of where the document has been sitting now costs one earned frame and leaves the hold standing, exactly as a missed frame already did. Only three such frames in a row, or losing the document altogether, ends the hold and starts a fresh one from the new position. The toler-

ances were widened at the same time: per-frame steadiness from 4.5 to 6.5 per cent of the frame diagonal, cumulative movement across the hold from 6 to 7 per cent, the stale window from 0.9 to 1.1 seconds, and consecutive missed frames tolerated from three to four.

Genuine detection is unchanged, and is now enforced at the moment of capture as well: the shutter only fires on a frame the detector has just judged settled, so a capture always lands on a document sitting still in the place it was held, after the full warm-up, the full 1.4 second hold and six earned detections.

Technical notes (Build 37)

- `lib/doc-scan-gate.ts` is the only application file changed. A frame is “displaced” when it lands outside either tolerance. A displaced frame decays one earned frame and leaves the anchor, the start time and the smoothed reference untouched, so a frame that returns to the settled position simply carries on; three in a row restart the hold. Auto-capture additionally requires the current frame not to be displaced, which is what preserves the slow-pan guard.
- Cumulative movement was widened to 7 per cent and no further. That is about 51 px of travel on the detection canvas; the slow-pan test moves at about 40 px per second and would capture at 8 per cent.
- Honest limit: a document creeping at roughly 20 px per second on the detection canvas can now complete a hold having moved about 28 px. That is a nearly still document and is the deliberate cost of making the hold usable on iOS.
- Verified: gate unit test 19 of 19. “Shaky hand still captures” (document still, every fourth frame placed wide) captures at 3.4 s; the same feed never captured under the Build 36 gate. “Constant wobble does not capture” and “a document that moves and stays serves a fresh hold” (capture 1.8 s after the move) both behave as intended, and the moving, flickering, full-frame and random-shape guards all still refuse. Type check clean, production build succeeds, auth and session smoke tests pass.
- Not verified on hardware: real iPhone and Samsung Browser devices are unavailable in the build environment, so this needs a spot-check on the reporting device after deploy. The `?debug=1` panel from Build 36 is there to read the figures off the handset.
- `lib/doc-scan.ts`, `components/scanner/scanner-screen.tsx` and `app/globals.css` are unchanged, so the Build 36 viewfinder, hints and diagnostics behave exactly as before.

Readable scans and a quicker grab (Build 38)

From Tracey’s testing on an iPhone 14 Pro Max: the grab felt slow and glitchy, and the saved image was not sharp enough to read small print when zoomed. Readability is the priority.

- Root cause of the soft images: capture came from a low-resolution camera preview stream. The scanner now requests a 4K stream (3840 x 2160), grabs the full-resolution frame at the shutter (capped at 16 megapixels) and, where the browser supports it (Chrome on Android), also takes a full still photo, used when it is more than 1.2 times larger and the same orientation.
- Straightening: detection runs on scaled copies of the full frame (1000 px and 640 px, plus full size up to 2600 px) and the corners are scaled back, then a bicubic perspective warp produces the page at its own resolution. It never upscales and caps at 3508 px on the long edge (A4 at 300 dpi). If detection fails on the still photo, it is retried on the full-resolution video frame, using the last well-framed live outline (under 1 second old) as a hint; if that also fails, the full-resolution photo is kept uncorrected and can still be cropped by hand.

- Encoding: capture and correction at JPEG 0.95, processed pages at 0.93 (was 0.9). Gallery and uploaded images use the same pipeline.
- Camera: on iPhone the main back lens is locked for the session so the phone does not switch lenses mid-hold; on Android continuous autofocus is requested where supported. The `?debug=1` panel shows the lens label and stream resolution on a camera line.
- Quicker grab: warm-up 1.2 s to 0.5 s (steady capture about 2.0 s, was 2.6 s); three wide frames in a row tolerated before the hold restarts (was two); a stray first frame re-anchors instead of locking the hold; the document need only fill 10 per cent of the frame (was 22), so the phone can be held further back, which helps phones with a long minimum focus distance. The 1.4 s hold, six earned frames, drift tolerances and the settled-frame capture rule are unchanged.
- Review screen: Save is a fixed gold footer bar below the scrolling panel (never overlapping the last field). Leaving after a capture asks “Are you sure you want to exit? Your unsaved changes will be lost.” (close button, back gesture, reload, or choosing a different source). The custom thumbnail is chosen by pinching, zooming and dragging the page under a 3:4 tile window, with Reset framing and Use whole page.
- Limits: the GoodBarber shell closing the whole webview cannot be intercepted by the page. The 25 MB per-file upload limit is unchanged; very long multi-page PDFs of high-resolution pages could approach it.
- Verified: gate unit test 21 of 21; imaging harness live detection 9 of 9 and straightening 5 of 5 (A4 from a 4K frame gives 1671 x 2613 px with 30 px fine print legible, versus 372 x 579 px from a 480 px stream). Type check, production build and auth smoke tests pass. Not verified on real iPhone or Samsung hardware, and the save bar, exit confirmation and thumbnail framer were build-tested only.

Auto-detect restored and a paged review card (Build 39)

- Auto-detect: the camera opens exactly as in Build 37 (rear camera, no resolution request) so the phone picks the same lens as when detection worked, then the same camera is raised to 1080p in place. The Build 38 4K request and the iPhone single-lens re-open are removed. Likely cause of the lost detection (hypothesis): the 4K request steering multi-lens phones to a different lens and a heavy live stream. The live detector now shrinks each frame in two smoothed steps. The full-resolution still at the shutter (Android Chrome) and full-resolution straightening are unchanged.
- Review card: one question per step with a progress bar, Back and Next: Check the page, Which vault?, Name this document, one step per vault field, Brightness and contrast, Save options. The last step’s button is the gold Save document. The scan stays visible above the card.
- Verified: gate unit test 21 of 21, type check, production build and auth smoke tests. Not verified on real iPhone or Samsung hardware; the `?debug=1` camera line shows the lens and resolution each phone delivers.

Fewer false grabs on busy backgrounds (Build 40)

- Why: up to Build 39 the detector only ever looked at the single largest edge outline in the frame. On a busy surface (wood grain, tiles, a keyboard, a patterned cloth) that outline is usually the background, so the page was missed, and a shape that did pass the shape checks could be stitched together from the page and something bright beside it.
- What changed (`detectCorners` in `lib/doc-scan.ts`): every closed outline is collected from three views of the frame (the scanning library’s own edge map, a light-against-dark view, and a gentle

edge view thickened so broken outlines close). Each outline covering at least 6 percent of the frame becomes a four-cornered candidate, must pass the existing shape checks, and then a new paper check (`looksLikePaper`):

- Every side must be a real paper edge: at least 75 per cent of 28 sample points along it step from a brighter inside to a darker outside, at most 10 per cent step the other way, and a thin dark line (tile grout, a seam) does not count as an edge.
- The middle of the shape must be clearly brighter than its surroundings, and no more than 25 per cent of the inside may be very dark (keys, a screen, stripes of shadow).
- The largest candidate that passes wins, so the text block printed inside a page never beats the page itself.
- Safety: a shape that fails the paper check is never returned, so it can never be captured. The old single outline is only offered when it is not well framed, purely to draw the guide outline. The capture gate, camera open, full-resolution capture, straightening and the “Use original” / “Use auto-cropped” review toggle are unchanged.
- Offline harness (single 360 px frames; the gate then needs about 2 seconds of steady detection):

Test set	Build 39	Build 40
Real photos with no document: shapes accepted	0 of 351	1 of 351 (a sunlight patch on a dark floor)
Documents placed on busy backgrounds: correct	6 of 120	64 of 120
Same set: partial (ID card found without its dark header band)	0	46
Same set: wrong shape	1	0
Same set: missed (manual shutter needed)	113	10
Real photos of documents: detected	7 of 19	9 of 19

- Known limitations: an ID card with a dark header band on a dark surface is often found without the band (“Use original” restores the full photo; extending the shape over the band was tested and rejected); white paper on a white desk still needs the manual shutter; a bright, paper-shaped sunlight patch on a dark floor can be accepted; 10 of 19 real document photos still fall back to the manual shutter.
- Verified: the shipped code was run in Chrome with the same OpenCV and scanning library versions the app loads, on 490 frames, and matched the analysis model on all 490 with no corner difference; median 14.8 ms per frame on the build machine (slowest 39.8 ms, against a 200 ms detection interval). Gate unit test 21 of 21, type check and production build pass. Not verified on a real iPhone or Samsung handset, and phone timing was not measured.

Preview thumbnails and no auto-trim (Build 26)

Two refinements to the save step, from testing feedback:

- No initial trim: the default crop now covers the whole frame, so an uploaded or captured image opens showing the complete picture with nothing trimmed. The user applies their own crop only if they want one.
- Preview thumbnails: when a document is saved, a small 3:4 preview is generated for scanned photos and for PDFs built from a scan. By default the whole page is fitted onto a white background so nothing is cut off. A “Set preview thumbnail” control opens a framing dialog with a draggable, resizable 3:4 frame so the user can choose exactly what the grid tile shows. Externally uploaded PDFs have no source image, so they keep the generic PDF icon. See files.md for how tiles render the thumbnail.